Fahrerzustand-Klassifikation

Von Burak Güldüz

Motivation & Problemstellung

Warum?

94% aller Verkehrsunfälle entstehen durch menschliches Fehlverhalten. Aktuelle Systeme wie Tesla Autopilot nutzen nur Drucksensoren im Lenkrad – das erkennt Ablenkung ohne Hände am Lenkrad nicht.

Das Problem

Wir haben Rohdaten aus dem VTI-Fahrsimulator, aber keine manuellen Ground-Truth-Labels. Wie generieren wir Labels objektiv, ohne zirkuläre Logik zu schaffen?

Ziele dieser Arbeit

- **Absolute Threshold Labeling:** Physikalisch interpretierbare, globale Schwellenwerte statt sessionabhängiger Perzentile
- **Temporal Persistence:** Event-Glättung zur Vermeidung von Label-Flackern
- **Multimodale Feature-Fusion:** 37 Features aus Fahrzeugdynamik, Blickverhalten und Physiologie
- **End-to-End Pipeline:** Von Rohdaten zur Zustandsvorhersage mit über 5 Millionen Trainingssamples

Datenerfassung & Modalitäten

Fahrzeugdynamik

- Geschwindigkeit, Beschleunigung, Jerk, Lenkwinkel, Gierrate, Spurposition

Eye-Tracking

- Blickrichtung (Varjo XR), Augenlidentöffnung, Off-Road-Ratio

Physiologie

- Herzrate, RR-Intervalle, HRV (SDNN, RMSSD)

Feature Engineering: 37 Features

Aggressive Driving (16)

- Beschleunigungsvariabilität
- Jerk (Ruckartigkeit)
- Lenkgeschwindigkeit
- Bremsfrequenz
- Headway-Verhältnisse

Inattentive (12)

- Spurabweichung (Lateral Std)
- Off-Road Gaze-Ratio
- NDRT Performance
- Augenlidentöffnung

Feature	Interpretation	Window
accel_long_mean	Durchschn. Längsbeschleunigung	50
accel_long_std	Variabilität der Beschleunigung	50
accel_long_max	Maximale Beschleunigung	50
accel_lat_std	Laterale Beschleunigungsvariabilität	50
jerk_long_mean	Durchschn. Ruck (Aggressivität)	50
jerk_long_max	Maximaler Ruck	50
jerk_lat_std	Laterale Ruck-Variabilität	50
speed_std	Geschwindigkeitsschwankungen	50
throttle_mean	Durchschn. Gaspedalstellung	50
throttle_changes	Frequenz von Gaspedalwechseln	50
steering_angle_std	Lenkvariabilität	50
steering_rate	Lenkgeschwindigkeit	50
yaw_velocity_std	Gierratenvariabilität	50
thw_mean	Durchschn. Headway (Zeit zum Vorderfahrzeug)	50
short_headway_ratio	Anteil kritischer Abstände (<2s)	50
brake_frequency	Bremsfrequenz	50
lateral_pos_std	Spurhaltevariabilität	50
lateral_pos_mean	Durchschn. Spurabweichung	50
gaze_heading_abs	Absolute Blickabweichung (horizontal)	-
gaze_pitch_abs	Absolute Blickabweichung (vertikal)	-
gaze_off_road	Binär: Blick off-road (>30° oder >20°)	-
gaze_off_road_ratio	Anteil off-road Blicke	50
eyelid_mean	Durchschn. Augenlidentöffnung	50
eyelid_low_ratio	Anteil geschlossener Augen (Müdigkeit, <0.3)	50
ndrt_error_rate	NDRT-Fehlerrate (falsch + timeout)	50
ndrt_total_attempts	NDRT-Aktivitätslevel	50
control_smoothness	Fahrflüssigkeit (inverse Steuer-/Gasvarianz)	-
hrv_sdnn	Heart Rate Variability (langfristig)	50
hrv_rmssd	HRV (kurzfristig, Stressindikator)	50

Geschwindigkeit: speed, speed_std

Umgebungsfahrzeuge: surround_actor_count, surround_min_dist, surround_avg_speed

Physiologie: hr_mean, hr_change

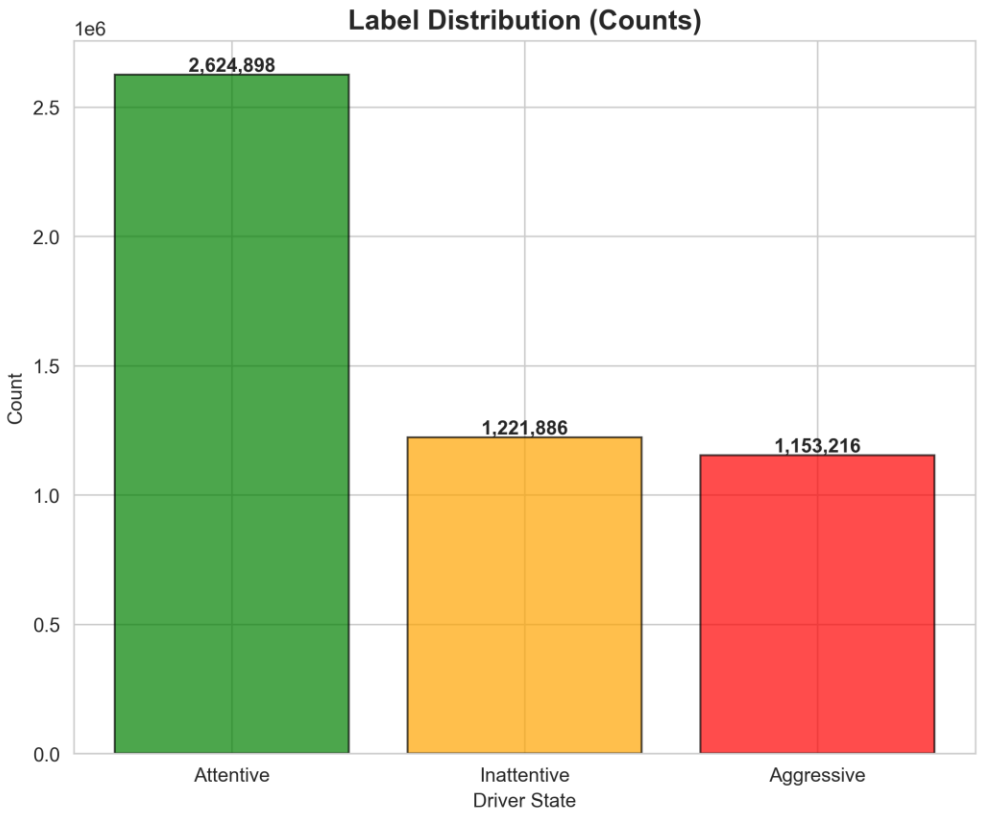
Label-Generierung: Absolute Threshold Labeling

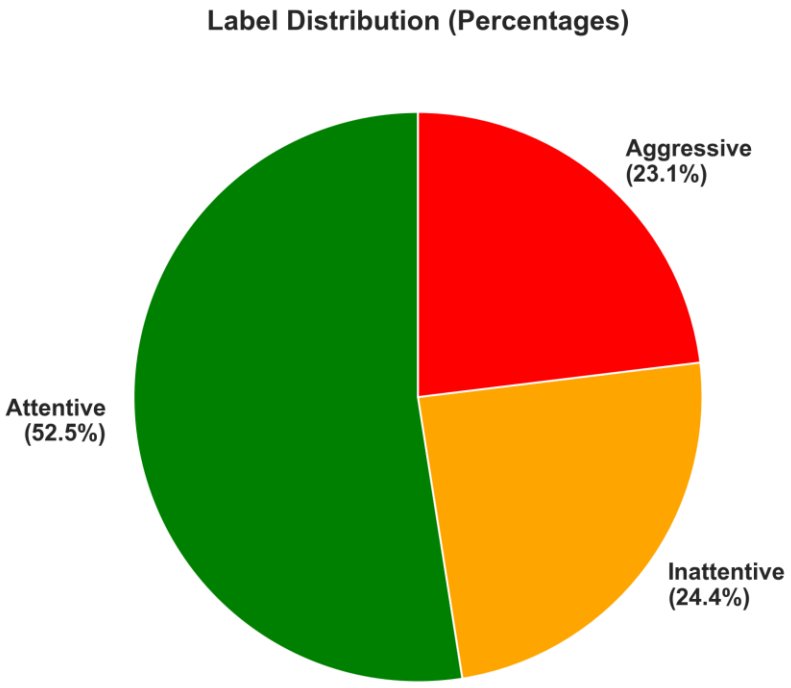
Wir vermeiden zirkuläre Logik durch interpretierbare, globale Schwellenwerte:

- **Phase 1:** Data-Driven Thresholds aus Perzentilen der Datenverteilung
- **Phase 2:** Kritische Events (Blick > 30° von Fahrbahn, Jerk > Schwelle)
- **Phase 3:** Akkumulatives Scoring – mehrere schwache Indikatoren = Zustand
- **Phase 4:** Temporal Persistence (z.B 2s Fenster) um Spikes zu glätten

```python
aggressive_events = np.zeros(len(features), dtype=bool)
        inattentive_events = np.zeros(len(features), dtype=bool)

        # CRITICAL EVENTS
        # These are events that are so critical that they override everything else.
        # Critical bevause inattentive driving is more dangerous than aggressive driving.
        # Example: Gaze of road looking at your phone might cause you to crash into a group of people.
        # Critical Inattention

        # |= is a bitwise OR.
        if 'gaze_off_road_ratio' in features.columns and 'gaze_off_road_high' in self.thresholds:
            critical_inatt = features['gaze_off_road_ratio'] > self.thresholds['gaze_off_road_high']
            inattentive_events |= critical_inatt

        if 'eyelid_low_ratio' in features.columns and 'eyelid_low_high' in self.thresholds:
            drowsy = features['eyelid_low_ratio'] > self.thresholds['eyelid_low_high']
            inattentive_events |= drowsy

        # Critical Aggression
        #
        if 'jerk_long_max' in features.columns and 'jerk_high' in self.thresholds:
            harsh_maneuver = features['jerk_long_max'].abs() > self.thresholds['jerk_high']
            aggressive_events |= harsh_maneuver

        # ACCUMULATIVE SCORING (Lower Priority)
        # Basically instead of doing if X is aggresive, we build a Temporal score over time.
        # So it becomes: if enough aggresive events happen, then the driver is aggressive.
        # Sum of weights is 1.0
        # Build aggression score
        aggression_score = np.zeros(len(features))
        if 'accel_long_std' in features.columns and 'accel_std_high' in self.thresholds:
            aggression_score += (features['accel_long_std'] > self.thresholds['accel_std_high']).astype(float) * 0.3
        if 'steering_rate' in features.columns and 'steering_rate_high' in self.thresholds:
            aggression_score += (features['steering_rate'] > self.thresholds['steering_rate_high']).astype(float) * 0.3
        if 'throttle_changes' in features.columns and 'throttle_changes_high' in self.thresholds:
            aggression_score += (features['throttle_changes'] > self.thresholds['throttle_changes_high']).astype(float) * 0.2
        if 'hr_mean' in features.columns and 'hr_elevated' in self.thresholds:
            aggression_score += (features['hr_mean'] > self.thresholds['hr_elevated']).astype(float) * 0.2

        # Build inattention score
        inattention_score = np.zeros(len(features))
        if 'lateral_pos_std' in features.columns and 'lateral_std_high' in self.thresholds:
            inattention_score += (features['lateral_pos_std'] > self.thresholds['lateral_std_high']).astype(float) * 0.4
        if 'ndrt_error_rate' in features.columns and 'ndrt_error_high' in self.thresholds:
            inattention_score += (features['ndrt_error_rate'] > self.thresholds['ndrt_error_high']).astype(float) * 0.3
        if 'control_smoothness' in features.columns:
            inattention_score += ((1 - features['control_smoothness']) > 0.5).astype(float) * 0.3

        # Apply accumulative thresholds
        agg_threshold = 0.5  # Need at least 50% of indicators
        inatt_threshold = 0.5

        aggressive_events |= aggression_score >= agg_threshold
        inattentive_events |= inattention_score >= inatt_threshold

        # Apply event persistence for temporal smoothing
        # Smoothing out shortterm spikes and noise.
        persistence_samples = int(event_persistence_seconds * sample_rate_hz) # Example: 2.0 seconds at 10 Hz is 20 samples
        if persistence_samples > 1:
            # Use rolling max to extend events
            aggressive_events = pd.Series(aggressive_events).rolling(
                window=persistence_samples, min_periods=1, center=True
            ).max().astype(bool).values

            inattentive_events = pd.Series(inattentive_events).rolling(
                window=persistence_samples, min_periods=1, center=True
            ).max().astype(bool).values

        # Assign labels Aggressive takes priority for safety of others.
        labels[aggressive_events] = 2  # Aggressive
        labels[inattentive_events & ~aggressive_events] = 1  # Inattentive (if not aggressive)
        # inattentive_events & ~aggressive_events element wise boolean logic as both are boolean arrays.
        # So assign inattentive only if the driver is inattentive but not aggressive as aggresive has higher
        # priority.

        # Store scores for analysis/debugging
        features['aggression_score'] = aggression_score
        features['inattention_score'] = inattention_score
```

Label Verteilung





Modellierung: Random Forest

Hyperparameter

- n_estimators: 200
- max_depth: 5
- min_samples_split: 30
- max_features: 'sqrt'
- class_weight: 'balanced'

Eigenschaften

- Interpretierbar
- Ensemble-Methode
- Robust gegen Overfitting
- Parallelisierbar für schnelleres Training

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=5,
    min_samples_split=30,
    min_samples_leaf=25,
    max_features='sqrt',
    random_state=random_state,
    class_weight='balanced',
    n_jobs=-1)
```

Random Forest: Ergebnisse

Test Accuracy & Metriken (Train/Test Split)

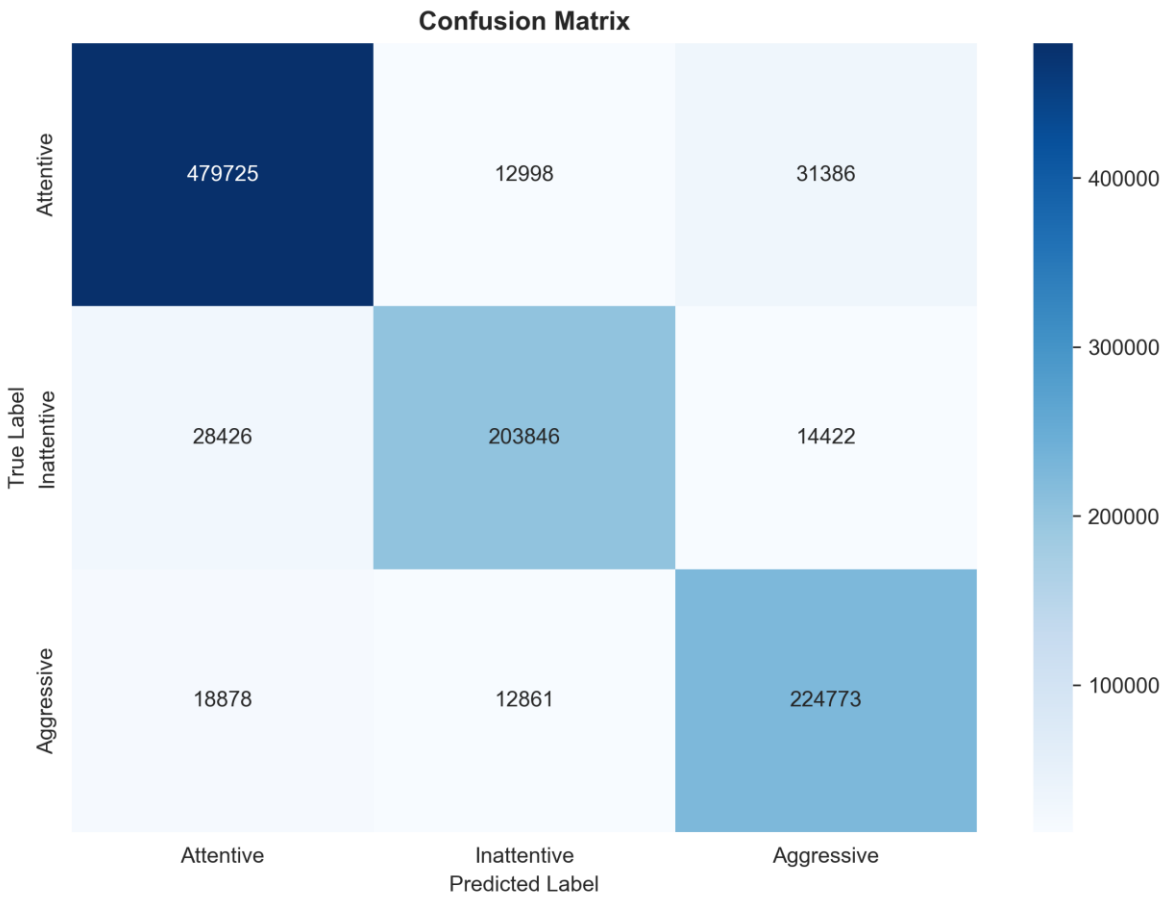
- **Train Accuracy:** 88.87% | **Test Accuracy:** 88.42%
- **Cohen's Kappa:** 0.8520 (starke Übereinstimmung)

Per-Class F1-Scores

- **Attentive:** 0.92 | **Inattentive:** 0.89 | **Aggressive:** 0.89

```
======================================================================
 === START: RANDOM FOREST TRAINING ===
======================================================================
RF Train Accuracy: 0.8887
RF Test Accuracy:  0.8842

RF CLASSIFICATION REPORT (test set)
-----------------------------------
              precision    recall  f1-score   support

           0     0.9102    0.9153    0.9128    524109
           1     0.8874    0.8263    0.8558    246694
           2     0.8307    0.8763    0.8529    256512

    accuracy                         0.8842   1027315
   macro avg     0.8761    0.8726    0.8738   1027315
weighted avg     0.8849    0.8842    0.8841   1027315


RF CONFUSION MATRIX (test set)
RF CONFUSION MATRIX (test set)
------------------------------
------------------------------
[[479725  12998  31386]
 [ 28426 203846  14422]
 [ 18878  12861 224773]]

RF Cohen's Kappa (test): 0.8129
```

Random Forest: Ergebnisse



Modellierung: LightGBM

Hyperparameter

- n_estimators: 200
- max_depth: 6
- learning_rate: 0.05
- subsample: 0.8
- GPU-Beschleunigung

Vorteile

- Gradient Boosting
- Schneller (GPU Training)
- Bessere Generalisierung
- Größere Tiefe möglich

```
lgbm_params = {
    'n_estimators': 200,
    'max_depth': 6,
    'learning_rate': 0.05,
    'subsample': 0.8,
    'colsample_bytree': 0.8,
    'min_child_samples': 20,
    'reg_alpha': 0.1,
    'reg_lambda': 1.0,
    'class_weight': 'balanced',
    'random_state': random_state,
    'verbose': verbose,
    'device': device
}
```

LightGBM: Ergebnisse & Vergleich

Test Accuracy & Metriken (Train/Test Split)

- **Train Accuracy:** 92.48% | **Test Accuracy:** 91.96% | **Train-Test**
- **Cohen's Kappa:** 0.8697 (sehr starke Übereinstimmung)

Per-Class F1-Scores

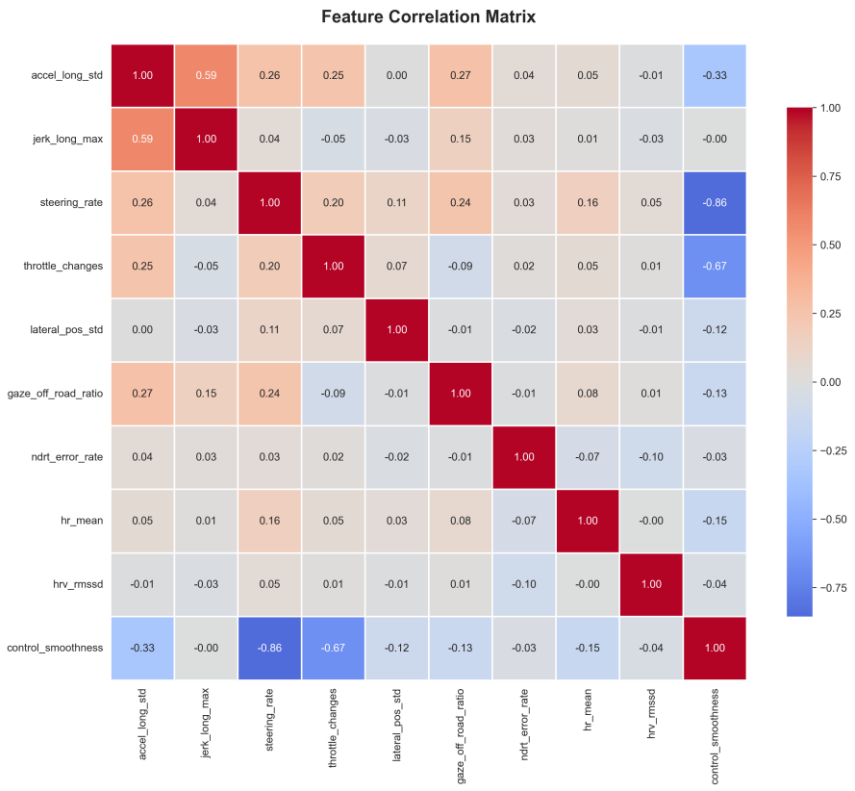
- **Attentive:** 0.94 | **Inattentive:** 0.90 | **Aggressive:** 0.90

```
Running session-based train/test split...
Train samples: 3972685
Test samples : 1027315

TRAIN / TEST RESULTS
============================================================
Train Accuracy: 0.9248
Test  Accuracy: 0.9196
Train-Test Gap: +0.0052

TEST SET CLASSIFICATION REPORT:
              precision    recall  f1-score   support

   Attentive       0.93      0.95      0.94    524109
 Inattentive       0.91      0.89      0.90    246694
  Aggressive       0.91      0.89      0.90    256512

    accuracy                           0.92   1027315
   macro avg       0.92      0.91      0.91   1027315
weighted avg       0.92      0.92      0.92   1027315

TEST SET CONFUSION MATRIX:
                   Attentive  Inattentive   Aggressive
      Attentive       497966       10004        16139
    Inattentive        22861      218700         5133
     Aggressive        15490       12935       228087

Test Cohen's Kappa: 0.8697
```

Statistische Merkmale: Datenübersicht

Datensatz-Statistiken

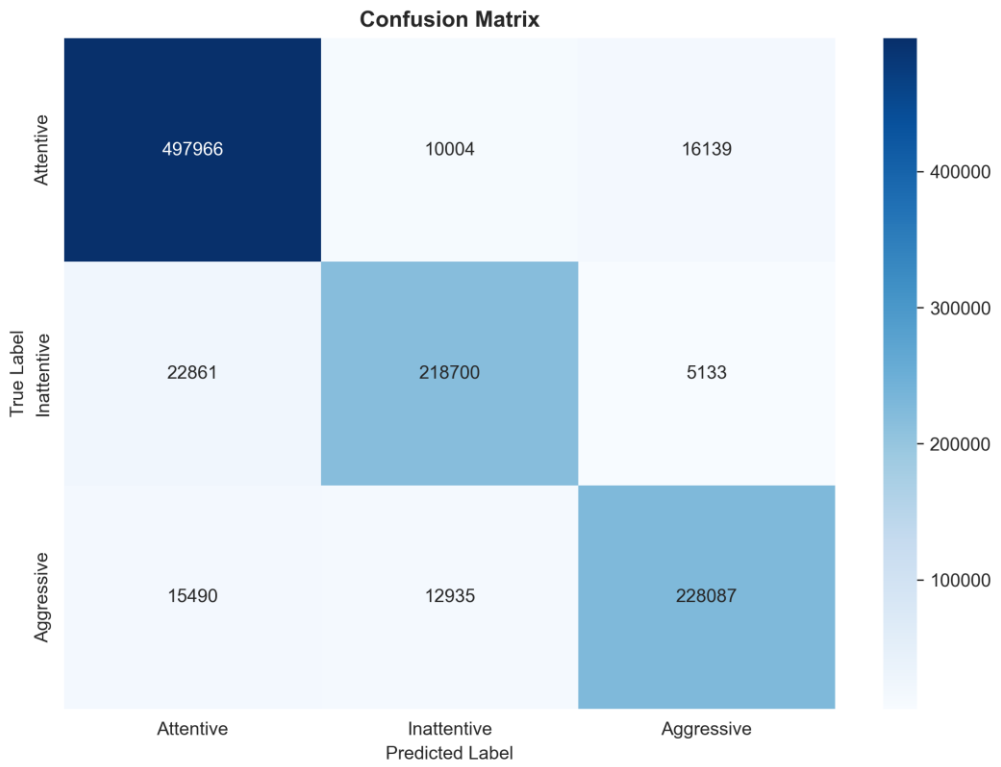
- **Gesamtsamples:** 5.037.534
- **Feature-Dimensionen:** 37
- **Trainsamples:** 4.010.219 (80%)
- **Testsamples:** 1.027.315 (20%)
- **Klassen:** 3 (Attentive, Inattentive, Aggressive)
- **Sampling Rate:** 50 Hz



Confusion Matrix & Klassifikation

Vorhergesagt
Attentive: 497.937 | 10.009 | 16.163
Inattentive: 22.882 | 218.734 | 5.078
Aggressive: 15.490 | 12.865 | 228.157

- **Observation:** Hauptdiagonale dominiert deutlich. Verwechslungen zwischen Inattentive und Aggressive sind gering (akzeptabel, da unterschiedliche Ursachen).



Top Features & Insights

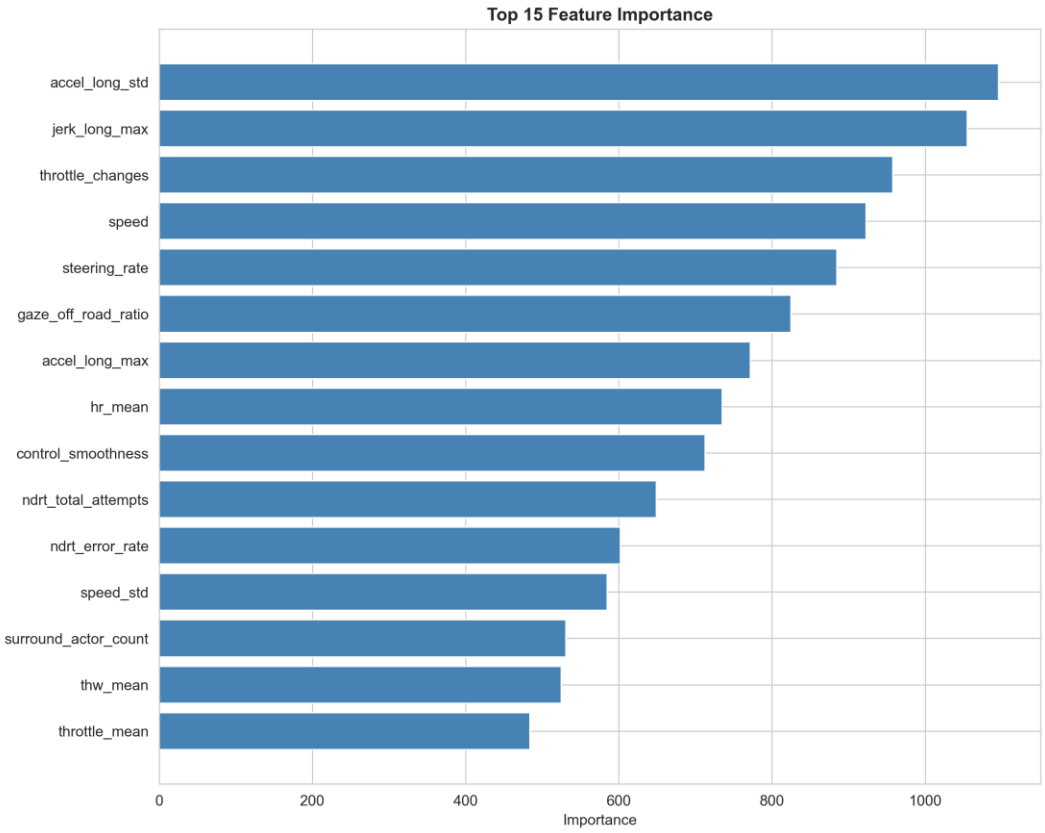
Wichtigste Indikatoren für Fahrerzustand

Für Aggressive Driving:

- Jerk (Beschleunigungsruckartigkeit)
- Steering Rate (Lenkgeschwindigkeit)
- Throttle Changes (Gaspedalwechsel)

Für Inattentive Driving:

- Gaze Off-Road Ratio
- Lateral Position Std (Spurabweichung)
- Eyelid Opening (Augenlidentöffnung)



Zusammenfassung & Ausblick

Schlüsselergebnisse

- Wir haben eine robuste Pipeline entwickelt, die multimodale Sensordaten integriert und Fahrerzustände mit 92% Genauigkeit klassifiziert, ohne manuelle Labels zu benötigen.

Zukünftige Arbeit

- Echtzeit-Integration in Fahrassistenzsysteme, Erweiterung auf Real-World-Daten, Unsupervised Learning (Clustering), und Takeover-Readiness für autonomes Fahren.